1 : Intro to Firewalling

Date	@3 février 2026
Professeur	De Fooz Pierre
To review	☐
Level	MASI 1
Q2	Network security
Type	Labo
Year	2025-2026

1. Filtering with Router Access Lists

Source	Destination	Port	Result	Router	Type
192.168.1.10 → 192.168.1.30	192.168.2.X	22 (ssh)	✅	R1 in (g0/0/0)	Extended
192.168.1.20 → 192.168.1.40	192.168.2.X	443 (https)	✅	R1 in (g0/0/0)	Extended
192.168.2.X	192.168.1.X	icmp	⛔	R1 in (g0/0/1)	Extended

To test `ssh` , we need to add an router and configure `ssh` on it in the second network.

```
ssh -l admin 192.168.2.3
```

> **Extended ACLs** should be placed **as close as possible to the source** of the traffic. This prevents unwanted traffic from crossing the network and wasting bandwidth before being dropped.

> **Standard ACLs** should be placed **as close as possible to the destination**. Since they only check the source IP address, placing them too close to the source might accidentally block legitimate traffic headed to other destinations.

💡 First put the biggest number of addresses first (attention for the deny)

ACL SSH for 192.168.1.10 → 192.168.1.30

Address range	Number of addresses	ACL
192.168.1.10 → 192.168.1.11	2 addresses	permit tcp 192.168.1.10 0.0.0.1 any eq 22
192.168.1.12 → 192.168.1.15	4 addresses	permit tcp 192.168.1.12 0.0.0.3 any eq 22
192.168.1.16 → 192.168.1.23	8 addresses	permit tcp 192.168.1.16 0.0.0.7 any eq 22
192.168.1.24 → 192.168.1.27	4 addresses	permit tcp 192.168.1.24 0.0.0.3 any eq 22
192.168.1.28 → 192.168.1.29	2 addresses	permit tcp 192.168.1.28 0.0.0.1 any eq 22
192.168.1.30	1 address	permit tcp host 192.168.1.30 any eq 22
others		deny tcp any any eq 22
		permit ip any any

To optimise more, we can do it in 4 lines with bigger ranges (with deny) :

Address range	Number of addresses	ACL
192.168.1.10 → 192.168.1.11	2 addresses	permit tcp 192.168.1.10 0.0.0.1 any eq 22
192.168.1.12 → 192.168.1.15	4 addresses	permit tcp 192.168.1.12 0.0.0.3 any eq 22
192.168.1.16 → 192.168.1.31	16 addresses	permit tcp 192.168.1.16 0.0.0.15 any eq 22
192.168.1.31	1 address	deny tcp host 192.168.1.31 any eq 22
others		deny tcp any any eq 22
		permit ip any any

⚠️ We need to put first the deny and then the permit with the 16 addresses

```
access-list 100 deny tcp host 192.168.1.31 any eq 22
access-list 100 permit tcp 192.168.1.16 0.0.0.15 any eq 22
access-list 100 permit tcp 192.168.1.12 0.0.0.3 any eq 22
access-list 100 permit tcp 192.168.1.10 0.0.0.1 any eq 22
access-list 100 deny tcp any any eq 22            -> block others ont s
sh

--- The following lines are to add after the ACL for HTTPS ---
access-list 100 permit ip any any                 -> permit others requ
ests

(config)# int g0/0/0                              -> on R2
(config-if)# ip access-group 100 in
-----------------------------------------------------------------
```

ACL HTTPS for 192.168.1.20 → 192.168.1.40

Address range	Number of addresses	ACL
192.168.1.20 → 192.168.1.23	4 addresses	permit tcp 192.168.1.20 0.0.0.3 any eq 443

Address range	Number of addresses	ACL
192.168.1.24 → 192.168.1.31	8 addresses	permit tcp 192.168.1.24 0.0.0.7 any eq 443
192.168.1.32 → 192.168.1.39	8 addresses	permit tcp 192.168.1.32 0.0.0.7 any eq 443
192.168.1.40	1 address	permit tcp host 192.168.1.40 any eq 443
others		deny tcp any any eq 443
		permit ip any any

```
access-list 100 permit tcp 192.168.1.24 0.0.0.7 any eq 443
access-list 100 permit tcp 192.168.1.32 0.0.0.7 any eq 443
access-list 100 permit tcp 192.168.1.20 0.0.0.3 any eq 443
access-list 100 permit tcp host 192.168.1.40 any eq 443
access-list 100 deny tcp any any eq 443       -> block others on https
access-list 100 permit ip any any             -> permit others requests

(config)# int g0/0/0                          -> on R2
(config-if)# ip access-group 100 in
```

ACL ICMP for 192.168.1.X

Address range	Number of addresses	ACL
192.168.2.0 → 192.168.2.255	256 addresses	deny icmp 192.168.2.0 0.0.0.255 any icmp
other		permit ip any any

```
access-list 101 deny icmp 192.168.2.0 0.0.0.255 any echo
access-list 101 permit ip any any

(config)# int g0/0/1                              -> on R1
(config-if)# ip access-group 101 in
```

---

2. Filtering with Linux Iptables

Connect first the 2 virtual machines, or 1 VM and the Windows pc.

- - - - - - - - - - - - - - Theory - - - - - - - - - - - - - -

1. Viewing and Managing Rules

Before adding rules, you need to see what is already there. The document outlines four main ways to list rules:

- **Basic List:** `# iptables -t filter -L` (Shows all rules in the filter table).
- **Verbose Mode:** Adding `v` shows the number of packets and bytes that have hit each rule.
- **Resetting Counters:** Adding `z` zeros out those packet/byte counters.
- **Line Numbers:** Adding `-line-numbers` is essential for identifying exactly where a rule sits (useful for deleting or inserting rules at specific positions).
- **Flushing:** The `-flush` command is used to delete all existing rules and start with a blank slate.

2. The Three Main Chains

The document asks about the difference between the chains shown in the diagram. In the **Filter** table, traffic is categorized into three paths:

- **INPUT:** Traffic coming from the network destined for the local machine.
- **OUTPUT:** Traffic generated by the local machine going out to the network.
- **FORWARD:** Traffic that is just passing through (the machine acts as a router/bridge).
- **PREROUTING / POSTROUTING:** Mainly used for NAT (Network Address Translation) to modify source or destination IPs.

3. Basic Command Structure

The general syntax provided for adding or removing rules is:
`iptables -t filter {-A | -D} {CHAIN} traffic -j {ACTION}`

- **A (Append):** Adds the rule to the very bottom of the list.
- **D (Delete):** Removes a specific rule.
- **j (Jump):** Defines the target action—either **ACCEPT** (let it through) or **DROP** (silently discard it).

4. Defining "Traffic" (Match Criteria)

To tell the firewall which packets to target, you define the "traffic" using specific flags:

- `i` (Interface In): The network card where the packet arrives (e.g., `eth0`).
- `o` (Interface Out): The network card where the packet leaves.
- `p` (Protocol): The type of traffic (TCP, UDP, or ICMP).
- `s` (Source): The IP address where the packet started.
- `d` (Destination): The IP address where the packet is going.
- `-dport` (Destination Port): The specific service port (e.g., 80 for HTTP, 443 for HTTPS, 22 for SSH).
- `j` (Jump): The action to take, usually **ACCEPT** (allow) or **DROP** (block).

- - - - - - - - - - - - - - Labo - - - - - - - - - - - - - -

Examples :

```
sudo su
iptables -t filter -L              -> to view the firewall
iptables -t filter -A INPUT -s 192.168.1.0/24 -j DROP
iptables -t filter -L
iptables -t filter -P INPUT DROP   -> put default policy into DROP and
not ACCEPT
iptables -t filter -L
```

💡 Use an private WiFi by changing in the settings to make an LAN.

1. Enabling IP Forwarding

Option A : Temporary (Lost after reboot) ← We use this one

You can directly change the value in the virtual file system:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Option B : Permanent (not here)

To ensure your router keeps working after a restart, edit the configuration file:

1. Open the file: `sudo nano /etc/sysctl.conf`
2. Find the line: `#net.ipv4.ip_forward=1`
3. Remove the `#` to uncomment it.
4. Save and exit, then apply the changes with: `sudo sysctl -p`

2. Identify interfaces and IPs

Before typing anything, you need to know your "Interface names" (like `eth0`, `ens3`, or `wlan0`).

- Run `ip link` or `ifconfig` to see your interface names.
- Check your DNS server IP by looking at `/etc/resolv.conf`.

3. The execution orde

```
# To become root
sudo su
```

▼ With eth0 :

```
### 1. Flush existing rules to start fresh
iptables -F
iptables -X

### 2. Allow Loopback interface
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

### 3. Allow Ping: Outgoing requests and incoming responses
iptables -A OUTPUT -o eth0 -p icmp -j ACCEPT
iptables -A INPUT -i eth0 -p icmp -m state --state ESTABLISHED,RELAT
ED -j ACCEPT

### 4. Allow Web Browsing (HTTP/HTTPS) and DNS
# Allow HTTP (80) and HTTPS (443) outgoing
iptables -A OUTPUT -o eth0 -p tcp -m tcp --dport 80 -j ACCEPT
iptables -A OUTPUT -o eth0 -p tcp -m tcp --dport 443 -j ACCEPT
# Allow DNS (Port 53) - Replace 8.8.8.8 with your DNS server IP
iptables -A OUTPUT -o eth0 -d 8.8.8.8 -p udp --dport 53 -m udp -j AC
CEPT
iptables -A INPUT -i eth0 -s 8.8.8.8 -p udp --sport 53 -m state --st
ate ESTABLISHED,RELATED -j ACCEPT

### 5. Allow updating system time (NTP)
iptables -A INPUT -i eth0 -p udp -m state --state ESTABLISHED,RELATE
D --dport 123 -j ACCEPT
iptables -A OUTPUT -o eth0 -p udp -m udp --sport 123 -j ACCEPT

### 6. Allow your machine to access external SSH servers
iptables -A OUTPUT -o eth0 -p tcp -m state --state NEW,ESTABLISHED -
-dport 22 -j ACCEPT
iptables -A INPUT -i eth0 -p tcp -m state --state ESTABLISHED --spor
t 22 -j ACCEPT

### 7. Allow your machine to be accessed via SSH (Incoming)
iptables -A INPUT -i eth0 -p tcp -m state --state NEW,ESTABLISHED --
dport 22 -j ACCEPT
iptables -A OUTPUT -o eth0 -p tcp -m state --state ESTABLISHED --spo
rt 22 -j ACCEPT

### 8. Allow your machine to receive a DHCP address
iptables -A INPUT -i eth0 -p udp -m state --state ESTABLISHED,RELATE
D --sport 67 -j ACCEPT
iptables -A OUTPUT -o eth0 -p udp -m udp --dport 67 -j ACCEPT
```

```
### 9. Protect Web Server & restrict SSH to specific IP range
# Allow all established traffic first
iptables -A INPUT -m state --state RELATED,ESTABLISHED -j ACCEPT
# Open Web Ports (80, 443) for everyone
iptables -A INPUT -p tcp -m state --state NEW -m multiport --dports
80,443 -j ACCEPT
# Limit SSH (22) to the range 192.168.1.10 - 192.168.1.20
iptables -A INPUT -p tcp -m state --state NEW -m tcp --dport 22 -m i
prange --src-range 192.168.1.10-192.168.1.20 -j ACCEPT
# Reject others with a specific ICMP error
iptables -A INPUT -j REJECT --reject-with icmp-host-prohibited

### 10. Set Default Policies (Block everything else)
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT DROP
```

▼ With ens33 :

```
### 1. Flush existing rules to start fresh
iptables -F
iptables -X

### 2. Allow Loopback and Established Traffic (Ensures you don't los
e connection)
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A OUTPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

### 3. Allow Ping: Outgoing requests and incoming responses
iptables -A OUTPUT -o ens33 -p icmp -j ACCEPT
iptables -A INPUT -i ens33 -p icmp -m state --state ESTABLISHED,RELA
TED -j ACCEPT

### 4. Allow Web Browsing (HTTP/HTTPS) and DNS
# Allow HTTP (80) and HTTPS (443) outgoing
iptables -A OUTPUT -o ens33 -p tcp -m tcp --dport 80 -j ACCEPT
iptables -A OUTPUT -o ens33 -p tcp -m tcp --dport 443 -j ACCEPT
# Allow DNS (Port 53) - Using Google DNS 8.8.8.8
iptables -A OUTPUT -o ens33 -d 8.8.8.8 -p udp --dport 53 -m udp -j A
CCEPT
iptables -A INPUT -i ens33 -s 8.8.8.8 -p udp --sport 53 -m state --s
tate ESTABLISHED,RELATED -j ACCEPT

### 5. Allow updating system time (NTP)
```

```
iptables -A INPUT -i ens33 -p udp -m state --state ESTABLISHED,RELAT
ED --dport 123 -j ACCEPT
iptables -A OUTPUT -o ens33 -p udp -m udp --sport 123 -j ACCEPT

### 6. Allow your machine to access external SSH servers
iptables -A OUTPUT -o ens33 -p tcp -m state --state NEW,ESTABLISHED
--dport 22 -j ACCEPT
iptables -A INPUT -i ens33 -p tcp -m state --state ESTABLISHED --spo
rt 22 -j ACCEPT

### 7. Allow your machine to be accessed via SSH (Incoming)
iptables -A INPUT -i ens33 -p tcp -m state --state NEW,ESTABLISHED -
-dport 22 -j ACCEPT
iptables -A OUTPUT -o ens33 -p tcp -m state --state ESTABLISHED --sp
ort 22 -j ACCEPT

### 8. Allow your machine to receive a DHCP address
iptables -A INPUT -i ens33 -p udp -m state --state ESTABLISHED,RELAT
ED --sport 67 -j ACCEPT
iptables -A OUTPUT -o ens33 -p udp -m udp --dport 67 -j ACCEPT

### 9. Protect Web Server & restrict SSH to specific IP range
# Open Web Ports (80, 443) for everyone
iptables -A INPUT -p tcp -m state --state NEW -m multiport --dports
80,443 -j ACCEPT
# Limit SSH (22) to the range 192.168.1.10 - 192.168.1.20
iptables -A INPUT -p tcp -m state --state NEW -m tcp --dport 22 -m i
prange --src-range 192.168.1.10-192.168.1.20 -j ACCEPT
# Reject others with a specific ICMP error
iptables -A INPUT -j REJECT --reject-with icmp-host-prohibited

### 10. Set Default Policies (Block everything else)
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT DROP
```

💡 Copier coller dans le terminal de la VM Linux

- Coller : `Ctrl` + `Maj (Shift)` + `V`
- Copier : `Ctrl` + `Maj (Shift)` + `C`

4. How to Save These Rules

Once you have run these commands, they are only active until the next reboot. To make them permanent on Ubuntu **or Debian**:

1. Install the persistent tool

`sudo apt update && sudo apt install iptables-persistent`

2. Save the current IPv4 rules

   `sudo iptables-save > /etc/iptables/rules.v4`

3. View the file

   `cat /etc/iptables/rules.v4`

If you want to keep the file but **do not** want the rules to apply automatically at startup :
`sudo mv /etc/iptables/rules.v4 /etc/iptables/rules.v4.backup`

---

3. Filtering with Windows Advanced Firewall

Windows Advanced Firewall Basics

The Windows Advanced Firewall (accessible via `wf.msc`) is a powerful tool for controlling network traffic. When a computer is part of a domain, these settings are typically managed centrally through Group Policy Objects (GPOs).

The firewall operates using three distinct **profiles** based on the network connection:

- **Domain Profile :** Active when the machine is connected to its corporate domain.
- **Private Profile :** Used for trusted networks (like a home or office LAN).
- **Public Profile :** The most restrictive mode, used for untrusted networks (like public Wi-Fi).

Task 1: Restrict OpenTftpServer Access

1. Download and install OpenTftpServer from SourceForge.net
2. Open Firewall → `Win + R` → `wf.msc`
3. Click on Inbound Rules > New Rule...
4. Rule Type → Select Program and browse to the OpenTftpServer executable.

   > OpenTftpServer is primarily used for transferring small files, such as firmware updates or configuration files, to network devices like routers, switches, and IP phones.

5. Action → Select Allow the connection.
6. Scope (Crucial): * Right-click your new rule > Properties > Scope tab.
   - Extended → Remote IP address, select These IP addresses > Add.
   - Choose This IP address range and enter:
     - From: `192.168.1.10`
     - To: `192.168.1.20`
7. Finish: Name it "Allow TFTP Subset" and click Finish.

Task 2: Allow ICMP (Ping) Only from Router

1. In Inbound Rules, click New Rule...
2. Rule Type → Select Predefined and choose File and Printer Sharing.
3. Specific Protocol: Find the rule for Echo Request (ICMPv4-In).
4. Scope: * Go to Properties > Scope tab.
    - Under Remote IP address, select These IP addresses and enter the specific IP address of your router (e.g., `192.168.1.1`).
5. Action: Ensure "Allow the connection" is selected.

Task 3: Disable Unnecessary Services on personal machine

1. Browse Rules: Look through the list of Inbound Rules.
2. Identify: Find entries related to services you don't use, such as:
    - Xbox (Gaming services)
    - Teredo (IPv6 tunneling)
    - Remote Assistance
3. Disable: Right-click each of these rules and select Disable Rule. (They will turn gray, meaning they are no longer active).